identifier_of Should Return std::string

Document #:	P3947R0 [Latest] [Status]
Date:	2025-12-15
Project:	Programming Language C++
Audience:	LEWG EWG

Reply-to:	Eddie Nolan <eddiejnolan@gmail.com>

Contents

§ 1 Status Quo

The current wording for [meta.syn] states:

Any function in namespace std::meta whose return type is string_view or u8string_view returns an object V such that V.data()[V.size()] equals '\0'.

[*Example 2:*

```
struct C { };

constexpr string_view sv = identifier_of(^^C);
static_assert(sv == "C");
static_assert(sv.data()[0] == 'C');
static_assert(sv.data()[1] == '\0');
```

- *end example*]

The relevant std::meta functions are symbol_of, identifier_of, and display_string_of, and their u8 equivalents.

§ 2 Issues with null-terminated std::string_view

§ 2.1 Teachability

The antipattern of contextually relying on std::string_view to have a null-terminated backing array is something we want to discourage.

For example, code like this is sometimes seen in the wild:

```
// precondition: sv is backed by a null-terminated byte string
void pass_to_c_api(std::string_view sv) {
  my_c_api(sv.data());
}
```

Since the precondition isn't enforced by the type system, it's easy to pass this function a string_view that isn't null-terminated and invoke UB within the C function that expects a C string.

This is the motivation for std::cstring_view ([P3655R3]).

Despite the fact that we teach users not to do this, however, we're now standardizing a std::string_view that provides users exactly the guarantees we are telling them we don't want them relying on.

§ 2.2 Tooling

If this was only a teachability problem, one could argue for the status quo by saying that the standard is not a tutorial. However, this situation also affects tooling. Compiler authors may want to implement a check that ensures that users are not accessing memory past

`size()` using a `string_view`. But with the current design, it could flag code that the standard has explicitly blessed, impeding its usefulness.

§ 2.3 Discoverability

Furthermore, if a user *does* need a null terminator, the interface doesn't make clear that one is present.

§ 2.4 Library UB

Finally, if a user attempts to access the null terminator by writing `str[str.size()]`, they violate the precondition of `std::string_view`'s `operator[]`. They instead need to write `str.data() + str.size()`. This is tricky.

§ 3 Consistently Returning Containers from Reflection Functions

In the reflection API, the `string_view`-returning functions give out a view to static storage data. Elsewhere in the API, however, we just return a container; for example, `members_of` returns `vector<info>` rather than `span<info const>`. It would be more consistent with `members_of` to have the `string_view`-returning functions return `string` instead.

§ 4 Returning `std::string` Instead Of `std::string_view`

This is what this paper proposes. It's a simple solution that solves the problems with the null-terminated `string_view` while restoring consistency with the design direction used by `members_of`. We already use `string` elsewhere in the reflection API in `data_member_options` and `std::meta::exception`.

§ 5 Compilation Performance and Memory Usage

Unfortunately, the current revision of this paper does not have concrete numbers to inform the following discussion. But implementers have had competing design concerns relating to this issue.

On the one hand, some implementers are primarily concerned with memory pressure. We believe that the original motivation for `members_of` to return `vector<info>` rather than `span<info const>` was because of this; we didn't want to cause the compiler to incur a persistent allocation every time it was invoked, especially since those allocations can't be

optimized away (since users rely on the lifetimes of the static storage data) whereas any corresponding increases in constant execution time can eventually be optimized further.

On the other hand, other implementers believe that the memory pressure concerns can be alleviated by more aggresive reclamation of data that isn't exposed by the constant expression, and claim that the performance differences between `std::string_view` and `std::string` for constant evaluation comprise multiple orders of magnitude, due to the fact that `std::string_view` is possible to model in the frontend whereas `std::string` is too complex to do so.

§ 6 Alternative Solutions

§ 6.1 Don't Null-Terminate the `std::string_view`

This is a more minimal change that addresses most of the concerns with the null-terminated `std::string_view`. However:

— It's inconsistent with `members_of` in that it's still giving out a view to static storage data

— It doesn't address the memory pressure concerns with giving out static storage data, described in the previous section

§ 6.2 Wait for `std::cstring_view` in C++29

The advantage of this approach is that we don't need to make a change in the C++26 time frame, and the number of users who'd be affected by this API change is minimal, since `std::string_view` and `std::cstring_view` are so similar.

The disadvantages are that it still is inconsistent with `members_of`, and still raises memory pressure concerns; and that we'd have standardized a problematic interface for C++26 with no guarantee that the `std::cstring_view` change would actually be viable in C++29.

§ 7 Ergonomics

The original motivation for returning `std::string_view` wasn't performance-related. It had to do with the fact that we wanted to guarantee the ability to write the following:

```
constexpr auto name = identifier_of(r);
```

This is possible with `char const*`, `std::string_view`, or hypothetically `std::cstring_view`, but not `std::string`.

However, when that design decision was made, we didn't have `define_static_string`. Now we do, so users can write:

```
constexpr auto name = define_static_string(identifier_of(r));
```

Which always works.

Unfortunately, for various reasons, the error messages if a user omits `define_static_string` can be terrible. (For further information, see the following Compiler Explorer pages from Barry Revzin: https://compiler-explorer.com/z/dxr5qPWvo (libc++ version) and https://compiler-explorer.com/z/heas35G77 (libstdc++ version)).

However, if, in the future, we manage to standardize a solution to non-transient constexpr allocation for `std::string` (such as the one proposed by [P3554R0]) then those concerns would be addressed.

§ 8 Wording

§ 8.1 21.4.1 Header <meta> synopsis

```
-  consteval string_view symbol_of(operators op);
+  consteval string symbol_of(operators op);
-  consteval u8string_view u8symbol_of(operators op);
+  consteval u8string u8symbol_of(operators op);
```

```
-  consteval string_view identifier_of(info r);
+  consteval string identifier_of(info r);
-  consteval u8string_view u8identifier_of(info r);
+  consteval u8string u8identifier_of(info r);
```

```
-  consteval string_view display_string_of(info r);
+  consteval string display_string_of(info r);
-  consteval u8string_view u8display_string_of(info r);
+  consteval u8string u8display_string_of(info r);
```

Any function in namespace `std::meta` whose return type is `string_view` or `u8string_view` returns an object `V` such that `V.data()[V.size()]` equals `'\0'`.

[*Example 2:*

```
struct C { };

constexpr string_view sv = identifier_of(^^C);
static_assert(sv == "C");
static_assert(sv.data()[0] == 'C');
static_assert(sv.data()[1] == '\0');
```

- end example]

§ 8.2 21.4.5 Operator representations

```
-  consteval string_view symbol_of(operators op);
+  consteval string symbol_of(operators op);
```

```
-  consteval u8string_view u8symbol_of(operators op);
+  consteval u8string u8symbol_of(operators op);
```

Returns: A ~~`string_view`~~`string` or ~~`u8string_view`~~`u8string` containing the characters of the operator symbol name corresponding to op, respectively encoded with the ordinary literal encoding or with UTF-8.

§ 8.3 21.4.6 Reflection names and locations

```
-  consteval string_view identifier_of(info r);
+  consteval string identifier_of(info r);
-  consteval u8string_view u8identifier_of(info r);
+  consteval u8string u8identifier_of(info r);
```

Returns: ~~An NTMBS~~A `std::string`, encoded with `E`, determined as follows:

…

— Otherwise, `r` represents a data member description (*T*,*N*,*A*,*W*,*NUA*) ([class.mem.general]); a ~~`string_view`~~`string` or ~~`u8string_view`~~`u8string`, respectively, containing the identifier N.

```
-  consteval string_view display_string_of(info r);
+  consteval string display_string_of(info r);
-  consteval u8string_view u8display_string_of(info r);
+  consteval u8string u8display_string_of(info r);
```

Returns: An implementation-defined ~~`string_view`~~`string` or ~~`u8string_view`~~`u8string`, respectively.

§ 9 Thanks

Thanks to Corentin Jabot, Barry Revzin, Daveed Vandevoorde, and Dan Katz, who discussed this topic in an email thread from which most of the information in this paper was paraphrased.

§ 10 References

[P3554R0] Barry Revzin, Peter Dimov. 2025-01-06. Non-transient allocation with vector and basic_string.
https://wg21.link/p3554r0

[P3655R3] Peter Bindels, Hana Dusikova, Jeremy Rifkin, Marco Foco, Alexey Shevlyakov. 2025-10-06. cstring_view.
https://wg21.link/p3655r3